

Introduction to Web Development & Architecture

Internet Infrastructure, Websites vs. Web Apps, Web Browsers & The Client-Server Model

1. The Global Network

TCP/IP, IP addresses, Domain Name System (DNS), and physical cabling connecting servers worldwide.

2. Client-Server Paradigm

Frontend user interface (Client) communicating asynchronously via HTTP to backend systems (Server).

3. Browser Runtime

Parsing HTML/CSS, DOM construction, JavaScript V8 engine, and layout/rendering pipelines.

1. How the Internet Works

Core Network Protocol

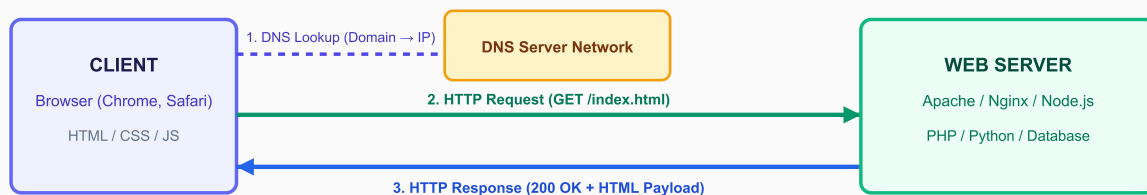
The **Internet** is a massive global network of interconnected computers communicating via standardized protocols (**TCP/IP**). The **World Wide Web (WWW)** is an information system operating on top of this physical network, where documents and resources are identified by URLs and linked via hyperlinks.

IP Addresses & Domain Names

- **IP Address:** Unique numerical identifier assigned to every device (e.g., IPv4: 192.0.2.1, IPv6).
- **Domain Name:** Human-readable aliases (e.g., example.com) mapped to IP addresses via the DNS lookup service.

HTTP / HTTPS Protocol

- **HTTP:** HyperText Transfer Protocol regulates how messages are formatted and transmitted between client and server.
- **HTTPS:** Secure HTTP encrypted with TLS/SSL encryption to prevent packet sniffing and data tampering.



2. The Client-Server Architecture

Separation of Concerns

Client-Side (Frontend)

Runs directly inside the end-user's web browser. Responsible for presentation, visual layout, and immediate user interactivity.

- **HTML5:** Defines document structure and semantics.
- **CSS3:** Manages layout, styling, colors, and responsive design.
- **JavaScript:** Controls DOM updates, logic, and dynamic behaviors.

Server-Side (Backend)

Runs on remote server hardware. Handles data persistent storage, business authentication, security, and API response generation.

- **Server Runtimes:** PHP, Node.js, Python, Java, Ruby.
- **Databases:** MySQL, PostgreSQL, MongoDB, SQLite.
- **Services:** Authentication, payment processing, file storage.

3. Websites vs. Web Applications

Architectural Shift

While both are delivered via web browsers, static websites and dynamic web applications differ fundamentally in complexity, interactivity, and state management.

Dimension	Static Website	Dynamic Web Application
Primary Focus	Information consumption & content presentation.	User interaction, task execution & data manipulation.
Content Generation	Pre-rendered HTML/CSS files stored on disk.	Generated dynamically on-the-fly via code and DB queries.
User Authentication	Rarely required (Read-only public content).	Core requirement (User accounts, sessions, permissions).
State Management	Stateless; each page request is independent.	Stateful; tracks user actions, shopping carts, and inputs.
Examples	Blogs, landing pages, documentation, portfolios.	Gmail, Plant Care Tracker, Trello, E-commerce portals.

4. Under the Hood: Web Browsers & Rendering Engine

Critical Rendering Path

A web browser is a complex client application that fetches web resources and converts raw HTML, CSS, and JS files into an interactive visual layout.

Browser Engine Modules

- **User Interface:** Address bar, back/forward buttons, bookmarks.
- **Rendering Engine:** Parses HTML/CSS (e.g., Blink, Gecko, WebKit).
- **JavaScript Engine:** Compiles and executes JS code (e.g., V8, SpiderMonkey).
- **Networking Module:** Manages HTTP requests, socket connections, and caching.

The Critical Rendering Path

1. **Parse HTML:** Construct the **DOM** (Document Object Model) tree.
2. **Parse CSS:** Build the **CSSOM** (CSS Object Model) tree.
3. **Render Tree:** Combine DOM + CSSOM to filter visible nodes.
4. **Layout (Reflow):** Calculate exact position and dimensions of elements.
5. **Paint:** Rasterize pixels to screen display.

5. HTTP Request Methods & Status Codes

Web Communication Protocol

Common HTTP Methods:

- GET: Retrieve data from server (No payload modification).
- POST: Submit new data to server (Creates resource).
- PUT / PATCH: Update existing server resource.
- DELETE: Remove resource from server storage.

HTTP Response Status Code Families:

2xx Success 200 OK, 201 Created
3xx Redirection 301 Moved Permanently
4xx Client Error 404 Not Found, 401 Unauthorized
5xx Server Error 500 Internal Server Error, 503 Unavailable

Day 0 Key Takeaways for Web Developers

Web development is fundamentally about building interfaces (Frontend) that consume services and store state (Backend) over a standardized packet-switched network (Internet). Mastering HTTP, DOM rendering, and client-server boundaries is the foundational step for all full-stack engineering.